

GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Modules and requirements

Company: Board&Byte **Group:** Group-8 **Date:** 7 November 2026

Name	Student No.	Name	Student No.
Tuğberk Akbulut	230104004070	Abdullatif Yılmaz	230104004031
Hasan Alperen Elmas	1801042604	Batuhan Yanar	220104004059
Ayşe Poyraz	230104004085	Berk Andaçgil	230104004066
Hasan Hüseyin Özdemir	230104004048	Muhib Taha Boy	230104004165
Selim Barut	220104004046	Umutcan Ocak	220104004903

1. Project overview

Çılgın Kurye is a multiplayer motorcycle courier game. Players play it in the web browser of their own phones, so there is nothing to install. Up to 8 players play at the same time in 4 teams, and the whole game runs on a server on one computer. Each player rides a motorcycle in a 3D city and carries packages from the team's store to delivery points. The team with the most points after 5 minutes wins.

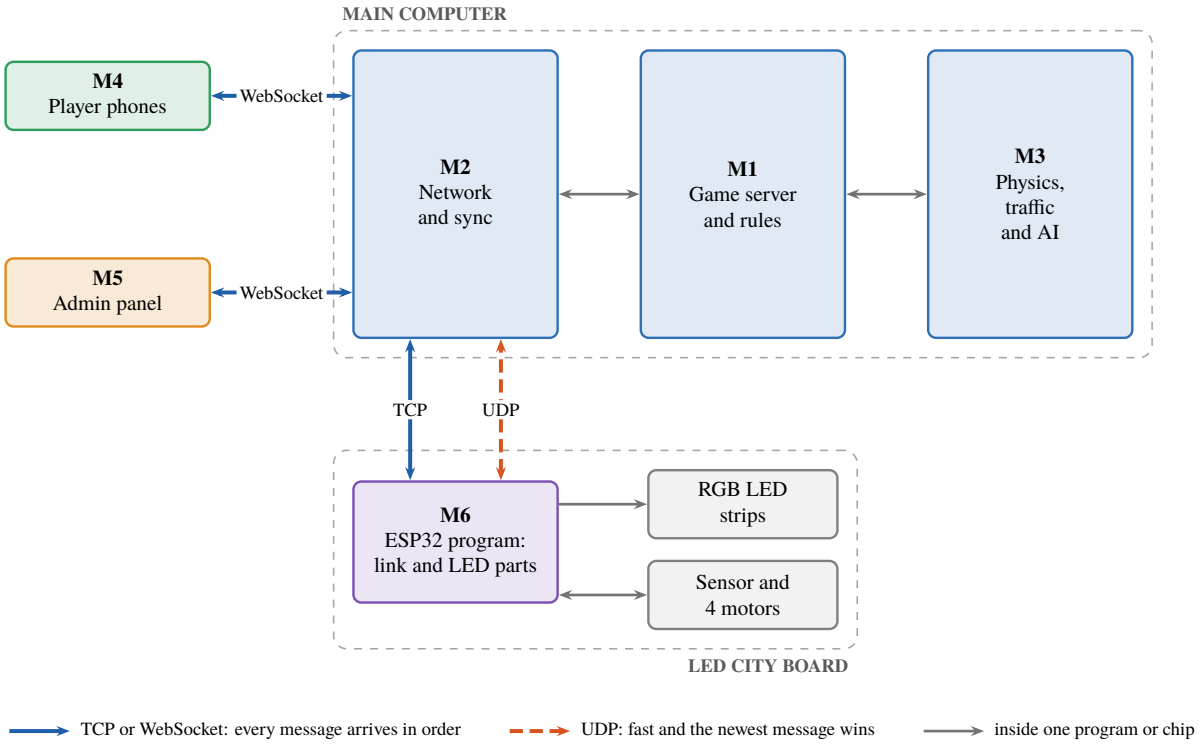
The special part of the game is that the phone has no map. To find the delivery point, players must look at the 60 × 60 cm city board on the table. LED strips in the roads of the board show all motorcycles, delivery points and obstacles in real time. The players can also hit the board with their hand to start an earthquake in the game. The admin computer is the control centre. During the game the admin changes parameters and sees each change in three places: in the 3D city on the phones, on the LED board and in live graphs. The computer, the phones and the board talk over Wi-Fi.



2. System architecture

The main computer runs one server program. Inside it, M1 runs the game, M3 moves the motorcycles, the traffic and the bots, and M2 does all the network work. This program makes every final decision, so all players see the same game. The phones (M4) only send their buttons and draw the 3D city. The admin page (M5) sends commands and shows graphs.

In the LED board there is one ESP32 (M6). It runs one Arduino program with two parts. The link part talks to the server, reads the motion sensor and drives the vibration motors. The LED part draws the LED frames. The figure below shows all six modules and the links between them. Each module chapter later has its own figure with only the data that goes into and out of that module.

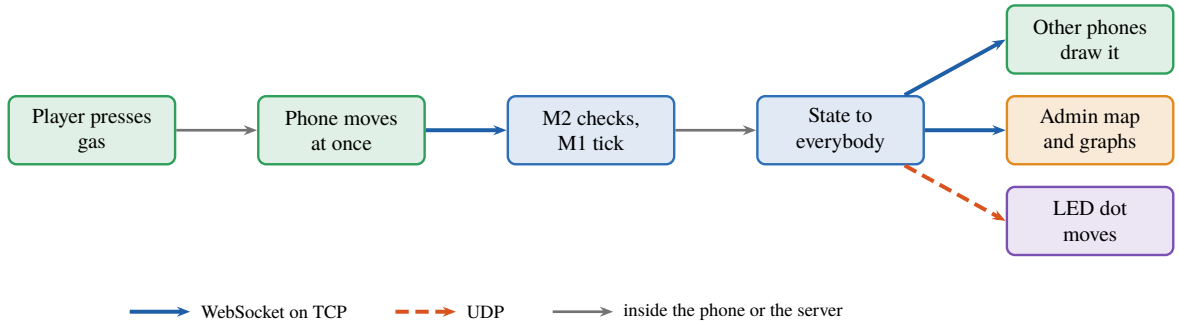


Which link uses which protocol:

A web browser can only open a WebSocket for talk in both directions, so the phones and the admin page use WebSocket, which runs on TCP. The LED board uses two channels. Messages that must never be lost go on TCP: the config, the vibrate commands for the motors, the earthquake events and the status report. Data that comes many times per second goes on UDP: the board state, the sensor samples and the delay probes. With UDP a lost packet is simply replaced by the next one, so the LEDs never stop to wait.

The path of one button press:

The next figure follows one press of the gas button. The phone moves its own motorcycle at once. The message goes to the server, M2 checks it, and M1 uses it in the next tick. Then the new state goes to every phone, to the admin page and to the board.



Five rules for the whole system:

First, there is one authority: only M1 changes the game state, and only inside a tick. Second, inputs are safe: phones send controls, never positions or scores, and M2 checks every message before M1 sees it. Third, all modules use shared contract files for the messages and one shared city plan, so the 3D city and the board always match. Fourth, each module can be built and tested alone with fake data from its neighbours. Fifth, the inputs of every tick go into a log, so a game can be played again exactly for testing.

3. System requirements

The table lists the requirements of the whole system. The last column shows the modules that meet each requirement.

No	Requirement	Description	Modules
R1	Multiplayer game	At most 8 players in 4 teams (at most 2 per team). A game lasts at most 5 minutes. AI bots fill empty places. The team with the most delivery points wins.	M1 M2 M3 M4
R2	Delivery cycle	The player takes a package from the store. The server chooses a delivery point by travel time with the current obstacles. The point blinks on the board in the player's colour. The player delivers and returns to the store. The near / far chance is set per player by the admin.	M1 M3 M4 M6
R3	Driving and controls	Free driving, lane changes and passing obstacles. No reverse, U turn by steering. Throttle, brake, left and right buttons on the phone.	M3 M4
R4	Collisions, falls, traffic	In a fast collision both players fall and wait a short penalty. NPC cars create real traffic. Hitting a car, a building or an obstacle fast makes the player fall. Penalty time and traffic density are admin parameters.	M1 M3
R5	No map on the phone	The phone shows only a compass arrow to the delivery point (or back to the store). No distance and no roads. Players must look at the board. "Find me" lights the player's LED white for 1 s.	M1 M4 M6
R6	Obstacles	Four types: heavy traffic, road work, accident, water puddle. On the board red always means "this road slows you down" and the pattern shows the type. A water puddle is light blue. Obstacles appear by themselves at random places during the game, never right next to a player, and each one ends after a set time. The admin can also add obstacles and can change the effect, the rate and the duration of each type.	M1 M3 M5 M6
R7	Earthquake mode	Starts by hitting the board by hand or with the admin button (vibration motors under the board). The sensor measures strength and approximate area. In the game the camera shakes, motorcycles fall and roads near the centre close. A shock wave spreads on the board. The strength is an admin parameter.	M1 M3 M4 M5 M6
R8	Physical LED board	60 × 60 cm, 6 roads in each direction (5 × 5 blocks), programmable addressable RGB LED strips on all roads, 3D printed buildings, four team stores in the corners with LEDs in the team colour. The ESP32 connects over Wi-Fi. The 3D city and the board are built from the same city plan.	M4 M6
R9	Live admin panel	All parameters are changed during the game. The admin sees the change in three places: 3D simulation, LED board and live graphs.	M1 M2 M5
R10	Network and measurement	Computer, phones and board talk over Wi-Fi. The admin can add delay, packet loss and a frame rate limit per player, and delay and loss on the board link. Ping, packet loss and frame rate are measured live.	M2 M4 M5 M6
R11	Engineering requirements	The main computer makes all game decisions (single authority). The phone only sends controls and sees its own motorcycle at once. Inputs and outputs of every module are defined, and each module can be developed and tested with mock data.	M1 to M6

4. Modules

The project has six modules on three sides: the server (M1 to M3), the phone and admin pages (M4, M5) and the physical board (M6). Every module has at least 3 members, and every member works in 2 or 3 modules. So a module can go on even if one member leaves.

Module	Name	Task	People
M1	Game Server and Rules	Single authority: game loop, rules, score, delivery, fall penalty, admin commands.	5
M2	Network and Synchronization	Phone, admin and board connections. Message encoding. Safe game state. Ping and network delay / loss settings.	4
M3	Physics, Traffic and AI	Motorcycle motion, collisions, falls, effects of obstacles and earthquakes, prediction on the phone. Routes and travel times, NPC traffic, bot drivers.	5
M4	Player Client and 3D City	3D city on the phone, controls, compass, prediction and correction.	3
M5	Admin Panel and Monitoring	Live parameters, map and obstacle tools, earthquake panel, graphs.	3
M6	LED Board and ESP32	Board connection, motion sensor and earthquake detection, vibration motors, LED animation engine and the physical board: base, LED strips, 3D printed city.	6

Member	M1	M2	M3	M4	M5	M6
Tuğberk Akbulut	X	X				X
Abdullatif Yılmaz	X		X	X		
Hasan Alperen Elmas			X			X
Batuhan Yanar			X		X	
Ayşe Poyraz		X			X	X
Berk Andaçgil	X			X	X	
Hasan Hüseyin Özdemir		X				X
Muhib Taha Boy			X			X
Selim Barut	X		X	X		
Umutcan Ocak	X	X				X
People	**5**	**4**	**5**	**3**	**3**	**6**

5. M1 Game Server and Rules

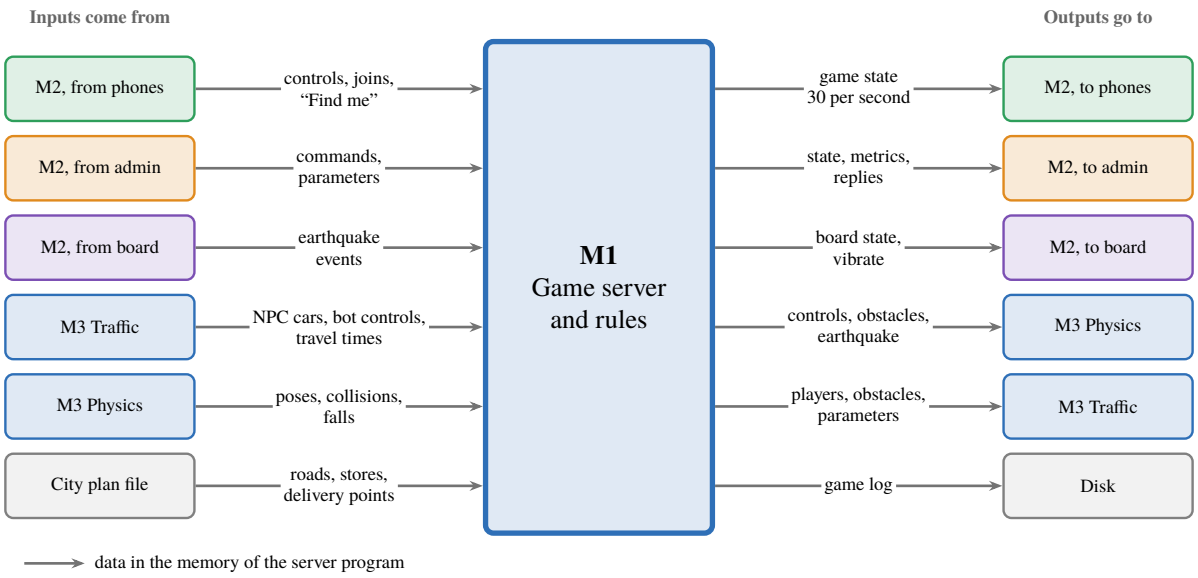
Team: Tuğberk Akbulut, Abdullatif Yılmaz, Berk Andaçgil, Selim Barut, Umutcan Ocak

5.1 Purpose

M1 is the brain of the game. It runs on the main computer and it is the only part that can change the game. It keeps the game clock and the one true game state. It knows every team, store, package, delivery point, score, fall, obstacle and earthquake. Phones only send their buttons. M1 decides what happens, and then everybody sees the same result.

Runs on	Technology	Requirements
Main computer, in one server program together with M2 and M3	TypeScript on Node.js, a fixed step game loop with 30 ticks per second, seeded random numbers, JSON files for the city plan and the game log	R1 R2 R4 R5 R6 R7 R9 R11

M1 has no network code. All its inputs and outputs are data in the memory of the server program. The figure shows which module gives M1 each kind of data and which module gets each result.



5.2 Requirements covered

M1 takes part in eight of the eleven system requirements. The last column says what will work when M1 is finished.

ID	Requirement	What M1 does for it	When M1 is done
R1	Multiplayer game	Lobby with 4 teams and at most 2 players in a team. Bots fill the empty places. A game is at most 5 minutes.	A full game runs from the lobby to the winner, with humans and bots together
R2	Delivery cycle	Automatic pickup at the store. Picks a delivery point by travel time, with a near or far chance per player	Every player always has a package or a point to go to, and points are fair
R4	Collisions and falls	Gives a penalty time after every fall that M3 reports	A fallen player waits and then rides on with the package
R5	No map on the phone	Puts the compass target and the "Find me" waiting time into the game state	The phone can show the arrow and the button without a map
R6	Obstacles	Keeps the one obstacle list that M3, M4 and M6 use. Adds random obstacles during the game and removes each obstacle when its time ends	A new or ended obstacle, automatic or from the admin, is seen everywhere in the same tick
R7	Earthquake	One path for a hand hit, the motors and the admin button. Road work and falls inside the radius	A hit on the board changes the game for everybody
R9	Live admin panel	Checks admin commands and sends each parameter to the module that uses it	Every parameter can be changed during the game
R11	Engineering requirements	Only M1 changes the game state. Every game can be played again from the log	The same log always gives the same game

5.3 Inputs and outputs

The table gives the content of each input and output. Value ranges are shown only for the main control values.

Direction	Data	From / to	Content
In	Controls	M2, from phones	Throttle and brake from 0 to 1, steer from −1 to 1, a control number. “Find me” comes as an event
In	Join	M2, from phones	Player key, name, team. M2 also reports when a player is lost or comes back
In	Admin commands	M2, from admin	Parameter changes, start, pause, resume, end, reset, new game code, obstacles, earthquake
In	Board events	M2, from board	Source (hand or motors), strength from 0 to 10, zone, confidence
In	Traffic	M3	NPC cars, bot controls, travel time table
In	Physics	M3	New motorcycle poses, collision and fall events
Out	Game state	M2, to phones and admin	Every tick: phase, time, scores, players, NPC cars, obstacles, earthquake. For each player also the compass target, the “Find me” waiting time and the number of the last control that M1 used
Out	Board state	M2, to board	Phase, players, delivery points, obstacles and earthquake in board coordinates
Out	Metrics	M2, to admin	Twice per second: the values of all admin graphs
Out	Game log	Disk	The seed and, for every tick, the controls, admin commands and board events

5.4 Main components

M1 is made of seven parts. Each part takes some data and gives a result to the next part or to another module.

Component	Takes	Gives
Game loop	The clock	Runs the steps of a tick in a fixed order, 30 times per second
Match and lobby	Joins, lost players, admin start and pause	Game phase, teams, bots in empty places
Delivery and scoring	Poses, travel times	Packages, delivery points, automatic deliveries, points
Penalties	Fall events from M3	Penalty time, then a stand up request to M3
Obstacles and earthquake	Obstacle parameters, admin obstacles, board events	The obstacle list with end times, earthquake centre and radius
Admin and parameters	Admin commands	Checked values sent to the owner module, ok or error replies
State builder and recorder	The world after the tick	Frozen game state, board state, metrics with a short history, the game log

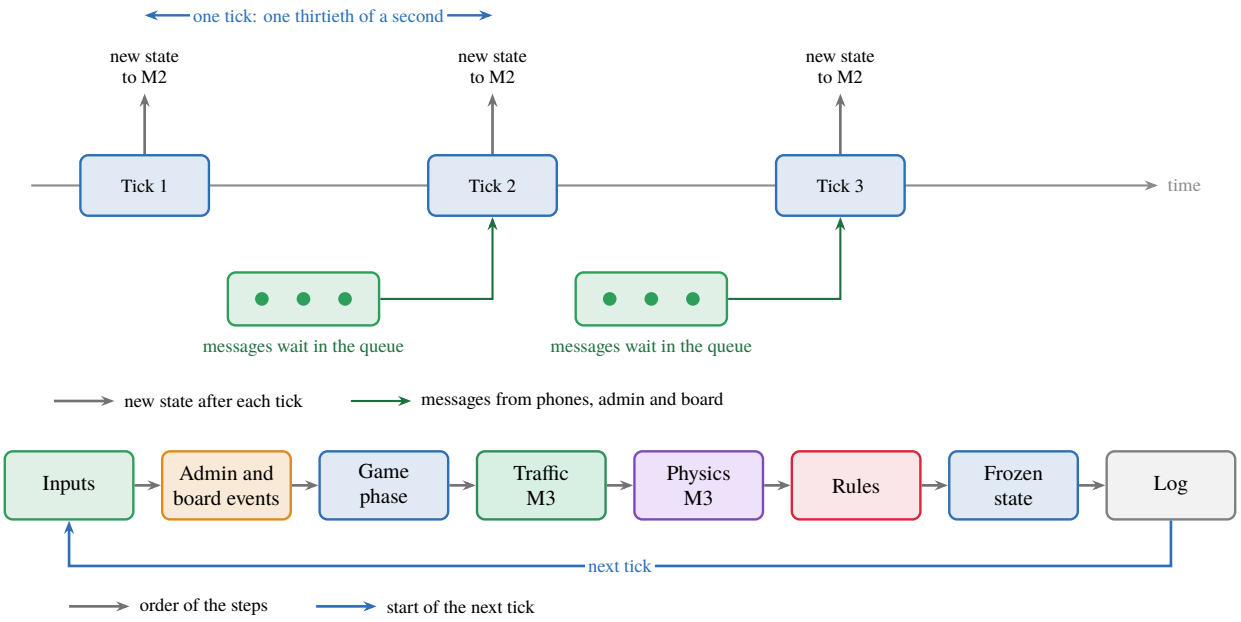
5.5 How it works

What a tick is:

The game world does not change all the time. It changes in small steps, like the pictures of a film. One step is called a tick, and the server makes 30 ticks in one second. Between two ticks, M2 only collects the messages from the phones, the admin and the board. In a tick, M1 uses all of them together, moves the world one step forward and makes a new state. Then M2 sends this state to everybody. So every player sees the same world after every tick.

The steps of one tick:

A tick always runs the same eight steps in the same order. First M1 takes the inputs that M2 has collected. Then it applies admin commands and board events, updates the game phase, asks M3 for traffic and bot controls, and then asks it to move all motorcycles. After that the game rules run. At the end M1 builds a frozen copy of the state for M2 and writes one line to the game log. Because the order never changes and all random numbers come from one seed, the same inputs always give the same game.

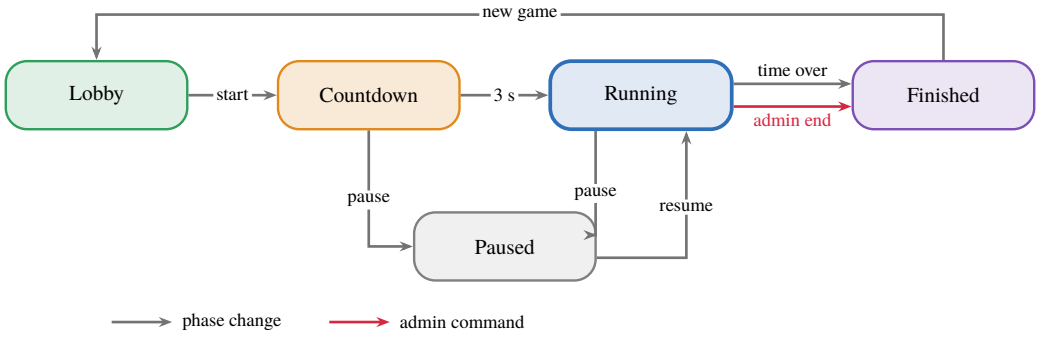


Controls inside a tick:

A phone sends its controls on every change and at least 10 times per second, but there are 30 ticks per second. So in some ticks a new control comes and in some ticks nothing comes. If nothing comes, M1 keeps the last control of that player. If more than one control comes, M1 uses the newest one for throttle, brake and steer. “Find me” is different. It is an event, not a value. If any message in the tick has a “Find me” press, the press counts, even when a newer message has no press. So a button press is never lost.

Game flow:

A game has five phases. In the lobby players join and choose a team. When the admin presses start, a 3 second countdown begins and nobody can move. Then the game runs for at most 5 minutes. The admin can end the game early. The admin can also pause and resume at any time. During a pause everything stops: the game time, the penalty times, the waiting time for lost players and the NPC cars. A pause during the countdown starts the countdown again after the resume. When the time ends, the team with the most points wins. If two teams have the same points, the game ends in a draw between them.



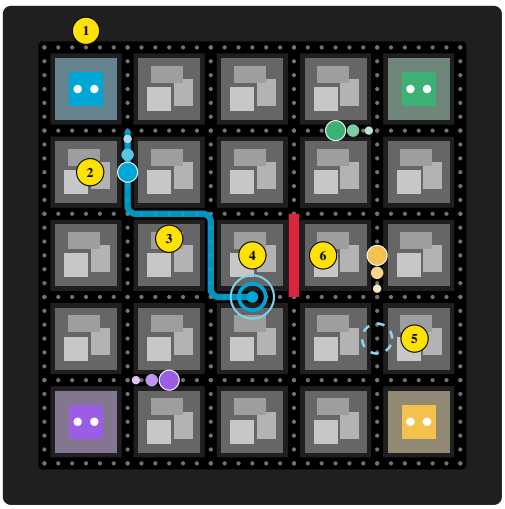
Players who leave and come back:

M2 sees when the link of a phone is lost and tells M1. M1 keeps the place of this player for 30 seconds. If the player does not come back in this time, a bot takes the motorcycle and plays on. If the player comes back later with the same player key, the bot gives the motorcycle back. The admin sets how many bots fill the empty places at the start.

Deliveries and points:

A player gets a package automatically when the motorcycle is inside the area of the own team's store. M1 then picks a delivery point for this player. M3 gives a table with the travel time from the store to every possible point. The times count the obstacles that are on the roads at that moment. The admin sets for each player whether the new delivery point appears near the store or far from it, and M1 picks the point with this setting. Two players never get the same point at the same time. A far point gives more points. Every delivery gets a base value, and the longer the expected travel time is, the bigger the bonus on top of it. The admin sets both the base value and the bonus. The points are fixed when the point is chosen, so a new obstacle later does not change them.

There is no delivery button. When the player comes to the delivery point and stops inside its area, M1 delivers the package automatically and adds the points. The size of the area is an admin parameter. A fall does not lose the package. The player only waits for the penalty time.



1 Team store in the team colour

2 Player with a package, a dot with a tail

3 Fastest route found by M3

4 Near delivery point, blinks in the player's colour

5 A far point, worth more points

6 Road work, so the route goes around

Earthquakes:

An earthquake can come from a hand hit on the board, from the vibration motors or from the admin button. All of them take the same path in M1. The board sends a strength from 0 to 10 and one of 9 zones. M1 uses the centre of that zone as the centre of the earthquake. A stronger hit makes a bigger earthquake. Motorcycles near the centre fall, and the roads near the centre close with road work for a limited time.

Obstacles:

M1 keeps one list of obstacles. Each obstacle has a type, a road segment, a lane, a position and an end time. During the game M1 adds new obstacles by itself at random places. The admin sets for each of the four types how often it appears and how long it stays. A new obstacle never appears right next to a player or right in front of one, so nobody falls without a chance to react. Obstacles from the admin and the road work from an earthquake also have a duration. When the time of an obstacle ends, M1 removes it from the list. While the game is paused, no new obstacle appears and the times stop. When the list changes, M3 computes the routes again, and M4 and M6 show the change.

Admin parameters:

Every parameter has a name, a group, a minimum, a maximum and an owner module. When the admin changes a value, M1 checks the limits first. Then it saves the value, sends it to the owner (for example grip to M3) and sends "ok" back to the panel. A wrong value gets an error and the old value stays.

5.6 Testing

M1 is tested alone, without the network and without the real M2 and M3. Small fake parts take their place inside the test and give fixed test data: scripted controls and joins, fixed poses and fall events, a fixed travel time table and recorded admin commands and board events. With this data we check every rule: the game phases with pause and end, the lobby and the bots, a lost player who comes back, the delivery cycle and the fair choice of points, two "Find me" presses in one tick, every admin command with good and wrong values, the earthquake radius, and the automatic obstacles with their rates, durations, distance to the players and the stop during a pause. Because all random numbers come from one seed, the same test data must always give exactly the same game state.

6. M2 Network and Synchronization

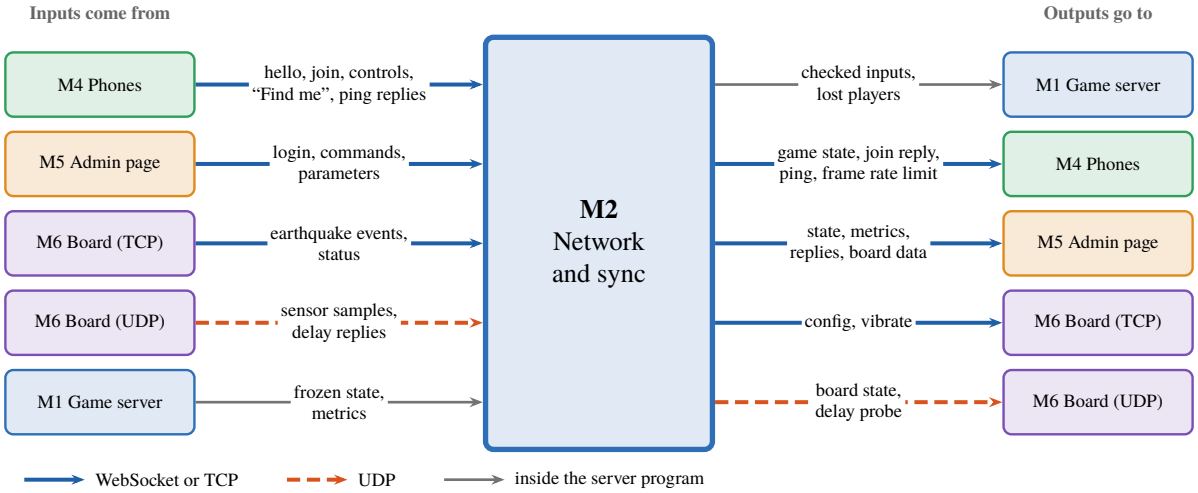
Team: Tuğberk Akbulut, Ayşe Poyraz, Hasan Hüseyin Özdemir, Umutcan Ocak

6.1 Purpose

M2 connects the phones, the admin page and the LED board to the game server. It does not make game decisions. It takes messages from outside, checks them and gives them to M1. After each tick it takes the state from M1 and sends it to everybody. M2 also makes sure that nothing from outside can change the game state directly. For the admin, it measures the network and it can make the network worse on purpose for tests.

Runs on	Technology	Requirements
Main computer, in the same server program as M1	TypeScript on Node.js, WebSocket for phones and admin, TCP and UDP for the LED board, MessagePack binary messages, QR code for joining	R1 R9 R10 R11

M2 is the only module with links to the outside. The figure shows every neighbour and the kind of link that each message uses.



6.2 Requirements covered

M2 takes part in four requirements. The last column says what will work when M2 is finished.

ID	Requirement	What M2 does for it	When M2 is done
R1	Multiplayer game	Up to 8 phones join with a QR code or a six digit game code, only with the browser	Players join in seconds without installing anything
R9	Live admin panel	Admin channel for commands, parameters, state, metrics and board data	The admin page controls the game from any computer on the game Wi-Fi
R10	Network and measurement	Ping and jitter for each phone, real loss on the board link. Added delay, jitter and loss for each player and for the board	The admin can make the network worse and see the effect live
R11	Engineering requirements	Inputs are checked and queued. Every phone gets the same frozen state of each tick	No phone can change the game by itself, and all players see the same game

6.3 Inputs and outputs

The table lists every message group with its link. All messages are MessagePack, a compact binary form of JSON.

Direction	Data	From / to	Content
In	Phone messages	M4, WebSocket	Hello with player key, name and game code, join with team, controls, “Find me”, ping replies
In	Admin messages	M5, WebSocket	Commands, parameter changes, requests for parameters and the game log
In	Board messages	M6, TCP and UDP	Earthquake events and status on TCP. Sensor samples and delay replies on UDP
In	State and metrics	M1	Frozen game state and board state every tick, metrics twice per second
Out	To M1	M1	Checked controls, joins, “Find me”, admin commands, board events, lost and returning players, network metrics
Out	To phones	M4, WebSocket	Game state 30 times per second, join replies, ping, frame rate limit
Out	To admin	M5, WebSocket	State 10 times per second, metrics, replies, board data
Out	To board	M6, TCP and UDP	Config and vibrate commands on TCP. Board state and delay probes on UDP

6.4 Main components

M2 is made of eight parts. Each one takes some data and gives a result to the next part.

Component	Takes	Gives
Web server and join	Page requests from phones and the admin	The phone and admin pages, the 3D models, the city plan, the QR code and the game code
Sessions and access	New links, player keys	One session for each phone and one admin. A player key keeps the same motorcycle after a reload
Codec and checker	Raw messages	Checked messages. A bad message gets an error reply. Too many or too large messages are dropped
Input queue	Checked inputs	The inputs of one tick for M1
State broadcaster	Frozen state from M1	The same encoded bytes for every phone, a smaller state for the admin and the board
Network emulator	Admin network settings	Added delay, jitter and loss for each player and for the board
Ping and metrics	Ping replies, order numbers	Ping, jitter, loss, clock offset and frame rate for the admin graphs
Board link	Board messages	One TCP channel and one UDP stream to the board

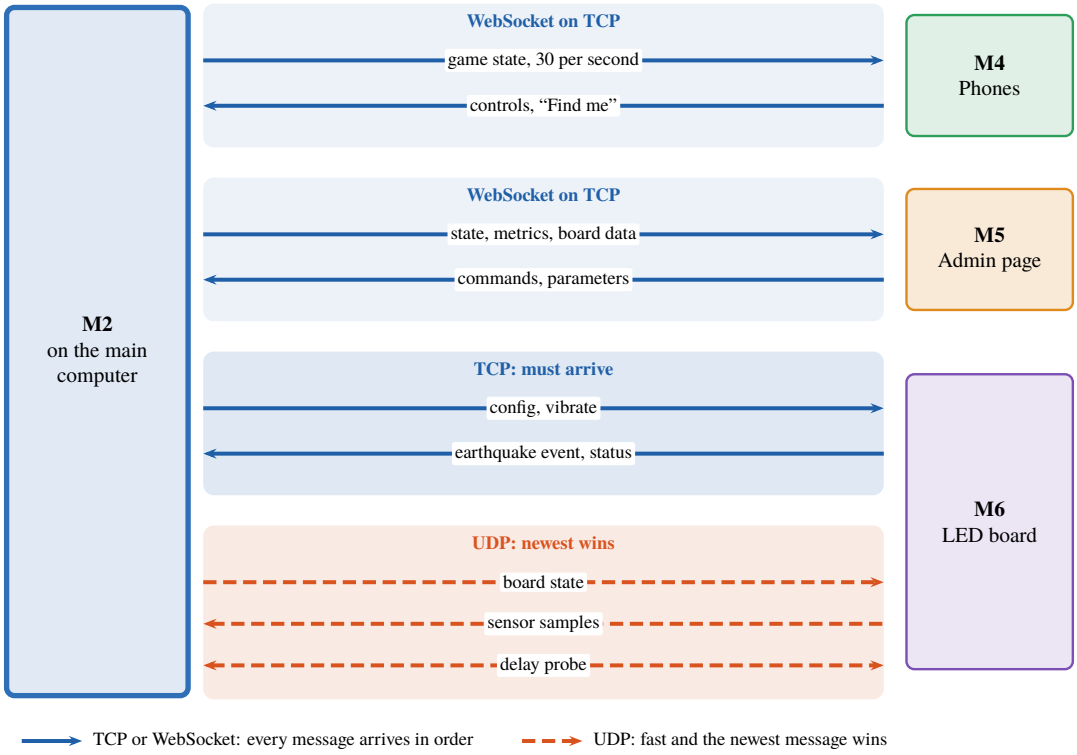
6.5 How it works

Which link uses which protocol:

A web browser can only open a WebSocket for talk in both directions, so phones and the admin page use WebSocket. WebSocket runs on TCP, so every message arrives and the order is kept. The LED board is not a browser, so it can use both TCP and UDP. On TCP it gets the messages that must never be lost: the config (player colours, brightness, power limit) and the vibrate commands. The board also sends earthquake events and its status report on TCP. The fast and repeated data goes on UDP: the board state, the sensor samples and the small delay probes. Here only the newest message is useful. If one UDP packet is lost, the next one replaces it. TCP would wait for the lost packet and the LEDs would stop for a moment, so UDP is the better choice for this data.

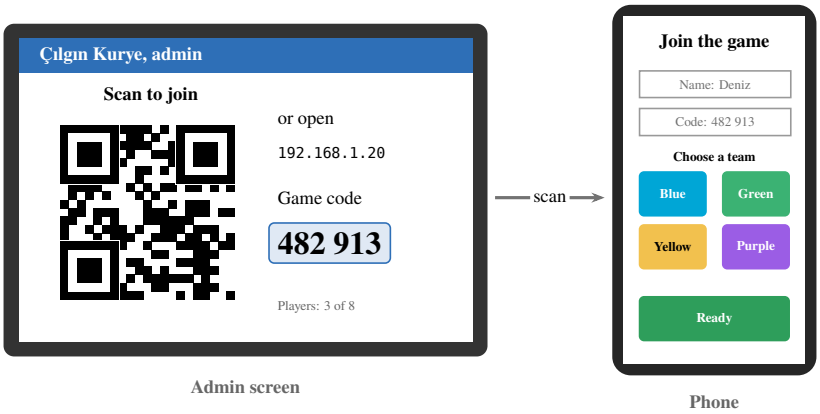
Messages on the board TCP channel:

WebSocket cuts the data into messages by itself, but plain TCP is only a stream of bytes. So each message on the board TCP channel starts with its length, and the other side reads exactly that many bytes. Small messages like a vibrate command are sent at once, without waiting to fill a bigger packet. Every message on both board channels has an order number, so old UDP packets can be dropped. Each start of the server makes a new session number. When the board sees a new session number, it starts its order numbers again, so new messages are not dropped as old ones.



Joining:

At the start the admin screen shows a QR code and a six digit game code. The QR code holds the IP address of the main computer and the game code. A player scans it, and the phone opens the game page from the main computer. A player without a camera types the address and the code. The first time, the browser saves a random player key. If the page reloads or Wi-Fi drops, the phone sends the same key and gets the same motorcycle back. The LED board first looks for the server on the local network. If it finds nothing, it uses a backup server address saved in the ESP32’s flash memory.

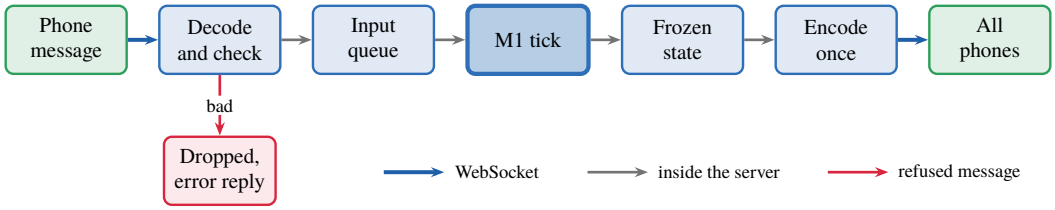


Keeping the game state safe:

Many phones send messages at the same time, but only M1 may change the game state, and only inside a tick. So M2 never writes to the state. A phone can only send controls. It can never send a position or a score. M2 decodes each message, checks its type and fields, and keeps every value in its range (for example steer stays between −1 and 1). Each session can only control its own player. A message with an old or repeated order number is dropped. Each session also has a limit on the number and size of its messages, so one phone cannot flood the server. Good inputs wait in a queue, and M1 reads the queue at the start of the next tick. At the end of the tick M2 encodes the frozen state once and sends the same bytes to all phones.

Network settings from the admin:

For tests the admin can make the network worse by hand. Each player has an up queue and a down queue, and the board link has the same pair. The admin sets three values for each queue: an added delay, a jitter that makes the delay change randomly, and a loss chance. The loss is used only on the state and control messages. Join, “Find me” and admin replies are never dropped, because the game would break without them. The admin can also set a frame rate limit for a phone and the board state rate, from 10 to 30 per second.



Measuring the network:

Twice per second M2 sends a ping with the server time. The phone answers with its own time, its frame rate and the number of the last state it got. From this M2 finds the round trip time, the jitter and the clock offset of the phone. On WebSocket a real Wi-Fi loss is not seen as a missing message, because TCP sends it again. It shows as a jump in the delay. So for phones M2 reports ping and jitter. On the board UDP stream a lost packet is really missing, so gaps in the order numbers give the real packet loss.

How much data:

We expect the full game state to be a few kilobytes. Sent 30 times per second to 8 phones, this is only a few megabits per second in total, which a normal 2.4 GHz Wi-Fi router can carry.

6.6 Testing

M2 is tested alone with test data instead of the real neighbours. A fake M1 sends a moving test state, scripted bot phones play the role of up to 8 real phones, and a small test program plays the board. With this setup we send good, broken and too large messages and check that no bad value reaches M1. We also check the load with 8 phones, how exact the added delay, jitter and loss are, that join and “Find me” are never dropped, reconnecting with the player key, and a server restart with the test board.

7. M3 Physics, Traffic and AI

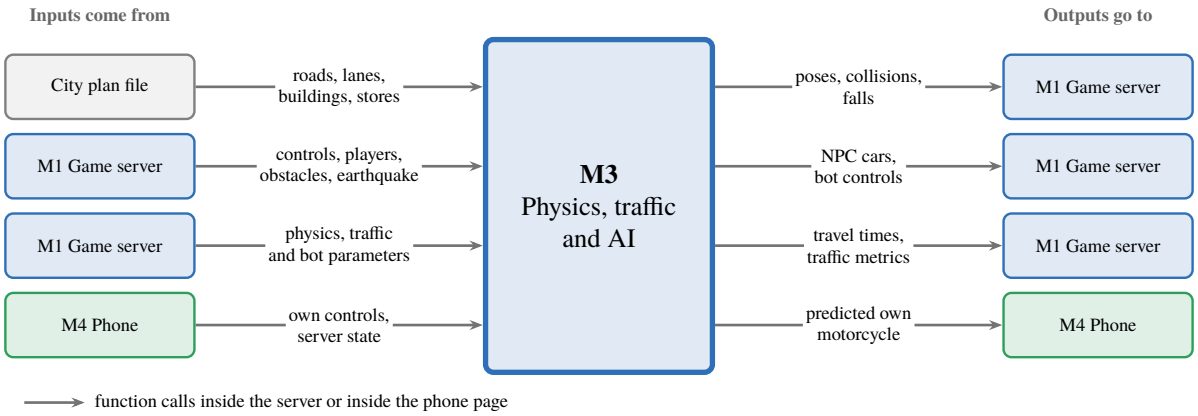
Team: Abdullatif Yılmaz, Hasan Alperen Elmas, Batuhan Yanar, Selim Barut, Muhib Taha Boy

7.1 Purpose

M3 moves everything on the roads. It has two parts. The physics part turns throttle, brake and steering into speed, direction and position. It finds collisions, decides when a motorcycle falls, and adds the effect of obstacles and earthquakes to the motion. The traffic part finds routes and travel times on the road network, moves the NPC cars that make the traffic, and drives the AI bots that fill the empty places in a game. A bot makes the same control record that a phone sends, so for M1 a bot and a human player look the same. The physics code also runs on the phone, where it guesses the player’s own motorcycle before the server answers.

Runs on	Technology	Requirements
Main computer inside the game server, called by M1 in every tick. The physics part also runs in the phone browser for prediction	TypeScript, a 3D physics engine that runs on the web, our own shortest path search (Dijkstra), a car following model for NPC cars, a state machine and path following for bots, seeded random numbers	R1 R2 R3 R4 R6 R7 R11

M3 has no network code. On the server it talks only to M1, and on the phone only to M4. The figure shows what goes in and what comes out.



7.2 Requirements covered

M3 takes part in seven requirements. The last column says what will work when M3 is finished.

ID	Requirement	What M3 does for it	When M3 is done
R1	Multiplayer game	Bots take the empty places and play like players	A game can start with only one human player
R2	Delivery cycle	Travel time table from each store, which M1 uses for fair near and far points	No team gets easier delivery points than another
R3	Driving and controls	Throttle, brake, left and right. No reverse. A U turn by steering in a small circle at low speed	The motorcycle drives, brakes and turns like an arcade game
R4	Collisions, falls, traffic	Collisions with buildings, barriers, NPC cars and motorcycles. A fast hit makes the motorcycle fall. NPC cars make real traffic	Nothing drives through a wall, fast crashes end in a fall, and the city is full of cars that never lock up
R6	Obstacles	Road work and accidents are solid. An accident has a slow zone. A puddle makes the motorcycle slide. Heavy traffic adds slow cars, and road work closes the road for routes and cars	Each obstacle type feels different when you drive, and cars and bots go around every obstacle
R7	Earthquake	Less grip inside the earthquake area that M1 sends	Riding in an earthquake is harder
R11	Engineering requirements	The same input always gives the same result. The server result wins, the phone only predicts. Talks only to M1 with plain data	The own motorcycle reacts at once and still matches the server, and M3 can be tested without the other modules

7.3 Inputs and outputs

M1 calls M3 in every tick: first the traffic part, then the physics part. M4 uses the physics code for one motorcycle.

Direction	Data	From / to	Content
In	City plan	Shared file	Roads, lanes, intersections, blocks, buildings, stores and possible delivery points
In	Controls	M1	Every tick: throttle and brake from 0 to 1, steer from −1 to 1, for humans and bots, and a speed factor for each player
In	Game data	M1	Players, obstacles with type, road, lane and position, the earthquake area and the motorcycles that fall
In	Parameters	M5, through M1	Top speed, grip, fall speed, grip in a puddle and in an earthquake, number of NPC cars, traffic effects, bot skill
Out	Poses	M1	Position, direction, speed, lean, sliding, fallen
Out	Events	M1	Collisions (with what and how fast) and falls
Out	NPC cars	M1, then M4, M5	Position, direction, speed, lane, size
Out	Bot controls	M1	The same record as a phone: throttle, brake, steer
Out	Travel times	M1	Seconds from each team store to every possible delivery point
Out	Traffic metrics	M1, then M5	Number of NPC cars, mean NPC speed, bot route changes
Out	Predicted pose	M4	The player's own motorcycle before the server answers

7.4 Main components

M3 has ten parts. The first four are the physics part and the next five are the traffic part. They all run on the server. The predictor runs on the phone.

Component	Takes	Gives
Motorcycle model	Controls, parameters	New speed, direction, lean and sliding
Colliders	City plan, obstacle list, NPC cars	Solid shapes, slow zones and slippery areas
Collision solver and fall judge	Moved objects, colliders	Pushed apart objects, collision and fall events
Earthquake effect	Earthquake area	Lower grip inside the area
Road graph	City plan	36 intersections and 60 roads in both directions, with lanes and turn costs
Route planner	Road graph, obstacles, traffic, parameters	Fastest routes and the travel time table, computed again when a cost changes
NPC traffic	Routes, players, car hits, number of cars	Moved, added and removed NPC cars
Bot drivers	Routes, the bot's motorcycle, game data	One control record for each bot in every tick
Traffic metrics	Cars and bots	Values for the admin graphs
Predictor	Own controls, server state	Predicted own motorcycle with a smooth correction

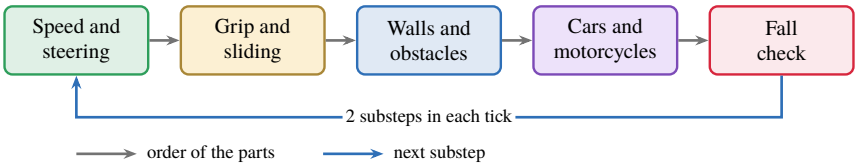
7.5 How it works

The two parts in one tick:

In every tick M1 first calls the traffic part. It moves the NPC cars and makes one control record for each bot. Then M1 calls the physics part with the controls of all players, humans and bots together. The physics part gives the car hits straight to the traffic part, so a hit car can stop or leave the road in the next tick.

One physics step:

The server runs 30 ticks per second, and the physics part runs 2 substeps in each tick. Small steps keep fast motorcycles from passing through thin walls. Each substep does five things in a fixed order, as the figure shows. Because the order is fixed, the same inputs always give the same result on the server and on the phone.

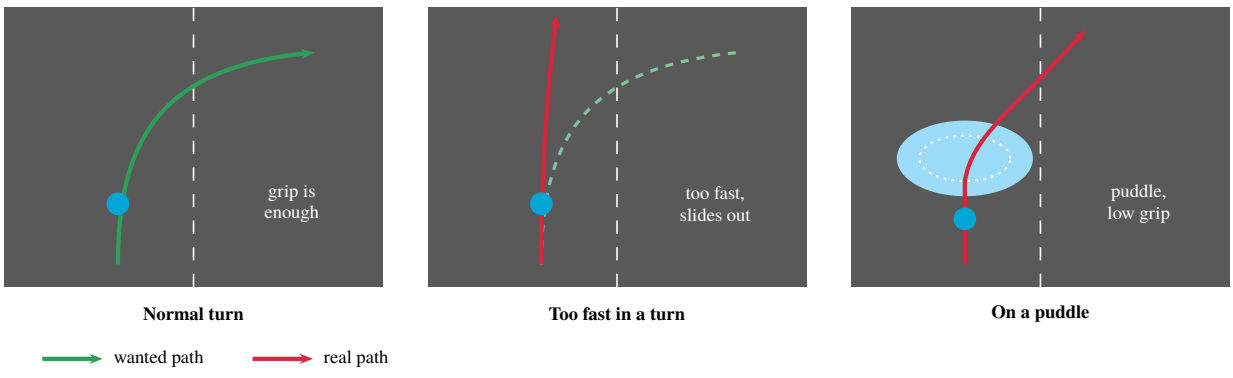


Driving:

The throttle adds speed up to the top speed, and the brake and air drag take speed away. There is no reverse. The steering changes the direction. At low speed the motorcycle can turn in a small circle, so a U turn is easy. At high speed the turn is wider. The model also gives a lean angle, which M4 uses to tilt the 3D motorcycle in a turn.

Grip and sliding:

In a turn the motorcycle needs a side force. M3 compares the size of this force with the grip limit. If the force is below the limit, the motorcycle follows the turn. If it is above the limit, the motorcycle slides out of the turn. We compare only the size, so a left turn and a right turn act the same. On a water puddle the grip limit is very low, so the motorcycle slides but does not slow down. Inside the earthquake area the grip is lower too.

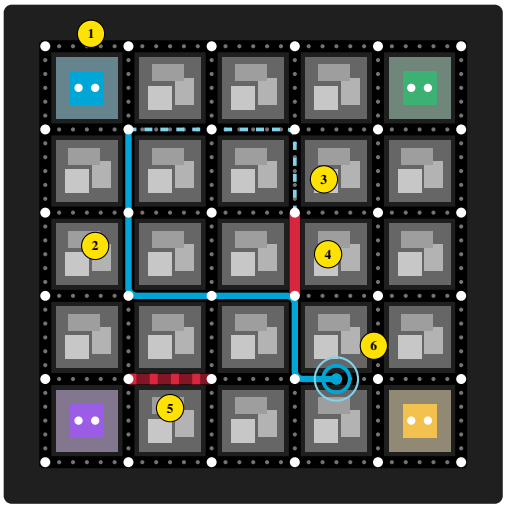


Collisions and falls:

Buildings, the outer wall and the road work barriers come from the city plan and the obstacle list as solid shapes. NPC cars and motorcycles have simple shapes too. A contact pushes the objects apart. If the hit speed is above the fall speed that the admin set, the motorcycle falls. When two motorcycles hit each other and their relative speed is above the limit, both of them fall. A fallen motorcycle is not solid, so it does not block the road for others. M1 gives the penalty time and later asks M3 to stand the motorcycle up again. An accident is solid in one lane and has a slow zone around it.

The road graph and routes:

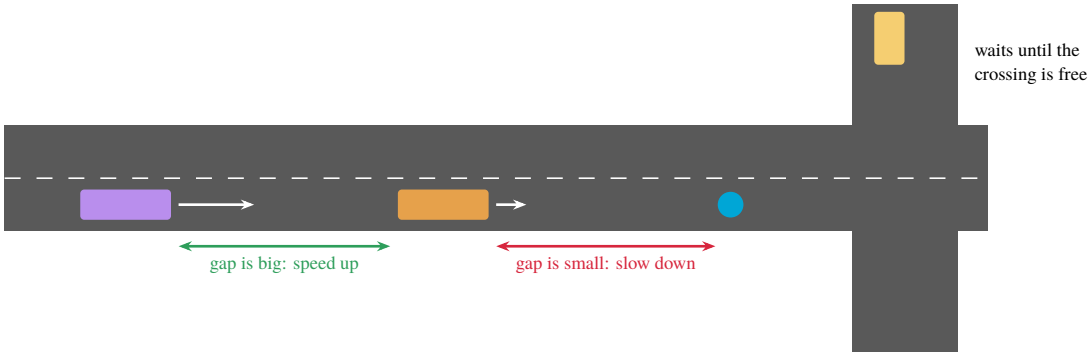
The city has 6 roads in each direction. Where two roads cross, there is an intersection, so the graph has 36 intersections. Between them there are 60 road segments, and cars can drive both ways. Each segment has a cost: the expected time to drive it. Turning left or making a U turn adds a small extra cost. Obstacles change the costs. Heavy traffic and an accident make a segment slower, and road work closes it. M3 uses Dijkstra's shortest path search to find the fastest routes. The graph is small, so one search is very fast. M3 computes the routes again whenever a cost changes: when an obstacle appears or ends, when the traffic on a road gets much heavier or lighter, or when the admin changes a route parameter. From the results it makes the travel time table: the time from each of the four stores to every possible delivery point. M1 uses this table to choose fair near and far points.



1 Start: the team store
2 Fastest route, found by Dijkstra
3 Shorter road, but closed
4 Road work: cost is infinite
5 Heavy traffic: cost is higher
6 Delivery point
◦ Intersection, a node of the graph

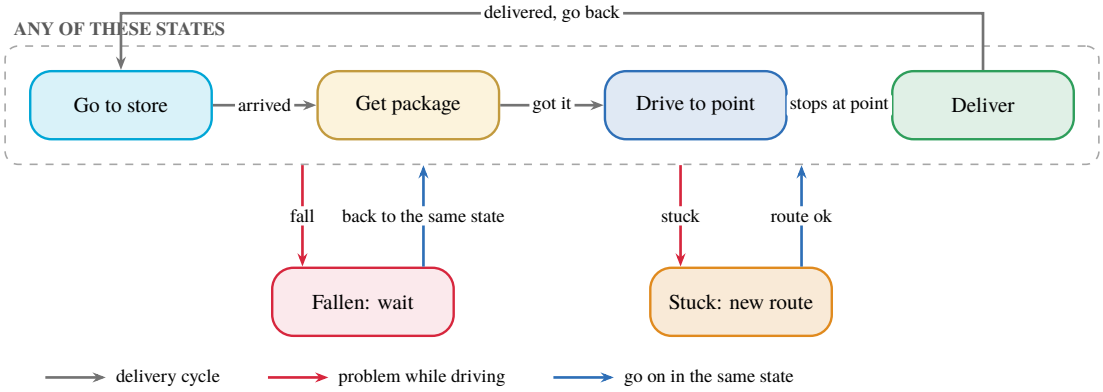
NPC traffic:

Each NPC car follows a lane and picks a new turn at each intersection. A car keeps a safe gap behind the car or the motorcycle in front of it. If the gap gets small, it slows down smoothly, and it speeds up again when the road is free. A car enters an intersection only when the intersection is free, so cars do not drive through each other. New cars never appear close to a player. If a car is stuck for too long, M3 removes it and adds it again somewhere else, so the traffic never locks up. The admin sets the number of cars, and heavy traffic obstacles add extra slow cars on their road.



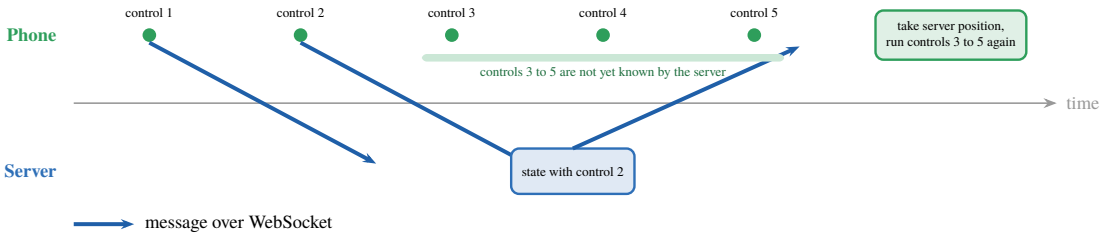
Bots:

Every bot has a small state machine that follows the delivery cycle, as the figure below shows. The bot drives to the store, gets the package, drives to the delivery point, stops there so the package is delivered, and then goes back. On the way it follows the route from the planner. It steers toward a point a little ahead on the path, so the motion looks smooth. A bot can fall or get stuck in any of these four states. If it falls, it waits for the penalty time and then goes on in the same state. If it is stuck, it asks for a new route and goes on in the same state. One "bot skill" value sets the top speed, the reaction time and some steering noise, so the admin can make bots weaker or stronger.



Prediction on the phone:

On the phone, the player must see the own motorcycle move at once, not after the delay of the network. So M4 runs a copy of the physics part for this one motorcycle. Each control gets a number and is kept in a list. When a server state arrives, the phone takes the real position from the server, throws away the controls that the server has already used, and runs the remaining controls again. If the new result is close to what the player sees, the difference is blended away in a short time. If it is far, for example after a crash, the motorcycle jumps to the right place.



7.6 Testing

M3 is tested alone with a small test program that plays the role of M1. It makes a world from the city plan and gives scripted controls, fixed player positions, a list of test obstacles and an earthquake area as test data. For the physics part we check a straight run and braking, a U turn, slides to the left and to the right, wall and car crashes, and the earthquake radius. For the traffic part we check routes with and without closed roads, equal travel times for all teams in a city that is the same on all sides, new routes when the traffic changes, many cars in a long run without a traffic lock, and bots that finish their deliveries. The same inputs and the same seed must always give exactly the same result. The predictor is tested with recorded server states and added delay and loss. A small test page shows a simple 3D view of the city from above, so one motorcycle can also be driven with the keyboard and the parameters can be changed with keys.

8. M4 Player Client and 3D City

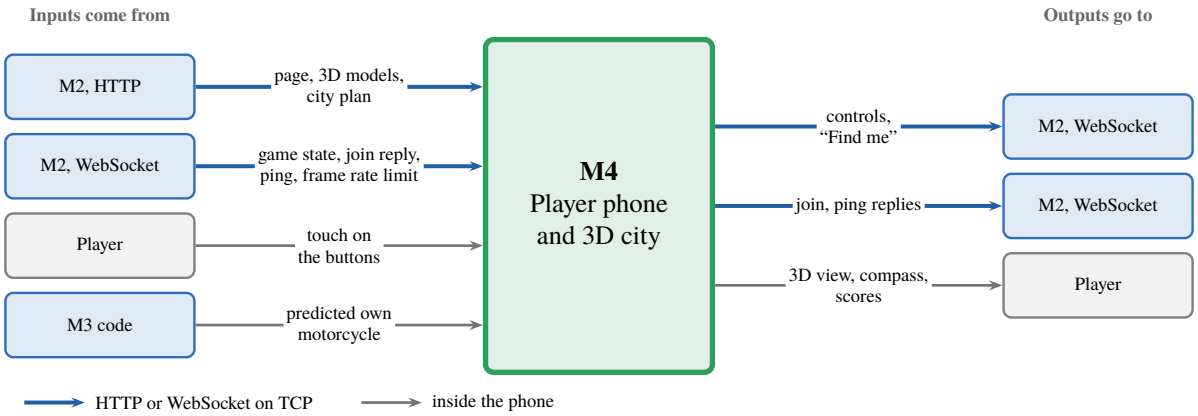
Team: Abdullatif Yılmaz, Berk Andaçgil, Selim Barut

8.1 Purpose

M4 is the web page on each player's phone. The player scans a QR code or types the game code, chooses a team, and drives a motorcycle in a low poly 3D city with buttons on the screen. There is no map on the phone. Only a compass arrow shows the direction of the delivery point or of the store. So players must look at the LED board on the table to find their way.

Runs on	Technology	Requirements
Player phone, only the web browser, nothing to install. The page comes from M2	TypeScript web page with Three.js for 3D, low poly models from one Blender file shared with the board buildings, MessagePack messages, the M3 physics code for prediction	R1 R2 R3 R5 R7 R8 R10 R11

The phone talks only to M2. It gets the page once over HTTP and then keeps one WebSocket open. The figure shows the data in both directions.



8.2 Requirements covered

M4 takes part in eight requirements. The last column says what will work when M4 is finished.

ID	Requirement	What M4 does for it	When M4 is done
R1	Multiplayer game	Join, team choice, timer and the scores of all teams	A player joins and plays from the browser only
R2	Delivery cycle	Package icon and a short message when the package is delivered	The player always knows if a package is on board
R3	Driving and controls	Gas, brake, left and right buttons	The motorcycle can be driven with two thumbs
R5	No map on the phone	Only a compass arrow, and the "Find me" button	The player must use the LED board to find the way
R7	Earthquake	The camera shakes, more near the centre	An earthquake is felt on every phone
R8	Same city as the board	The 3D city is built from the shared city plan and the same models	The same building is found on the phone and on the board
R10	Network and measurement	Frame rate limit from the admin, frame rate report	The admin sees how each phone runs
R11	Engineering requirements	Own motorcycle is predicted at once and corrected by the server	Driving feels direct even on a slow network

8.3 Inputs and outputs

The page sends its controls on every change and at least 10 times per second, and it gets a new state 30 times per second.

Direction	Data	From / to	Content
In	Page, models, city plan	M2, HTTP	When the page loads: the program, the 3D models and the city plan
In	Game state	M2, from M1	Players, NPC cars, obstacles, scores, compass target, "Find me" waiting time, number of the last control that the server used
In	Join reply, ping, net config	M2	Player, team, colour, server time, frame rate limit
In	Touch	Player	Buttons
Out	Controls	M2, then M1	Throttle, brake, steer, a control number
Out	Join, "Find me"	M2, then M1	Player key, name, game code, team, "Find me" request
Out	Ping reply	M2	Phone time, frame rate, last state received

8.4 Main components

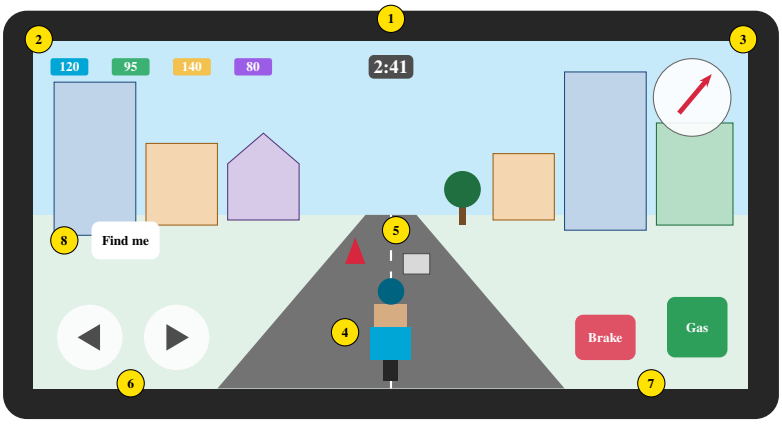
The page has five parts. The network client is the only part that talks to M2.

Component	Takes	Gives
Screens and HUD	Game state, join reply	Join, lobby, countdown, game and end screens, timer, scores, compass
Controls	Touch on the buttons	The control record and the button events
Network client	Messages from M2, the saved player key	Decoded state for the other parts, reconnect after a lost link
Prediction	Own controls, server state	Own motorcycle at once, with a smooth correction
3D city and scene	City plan, models, game state	The 3D picture with motorcycles, cars and obstacles

8.5 How it works

The game screen:

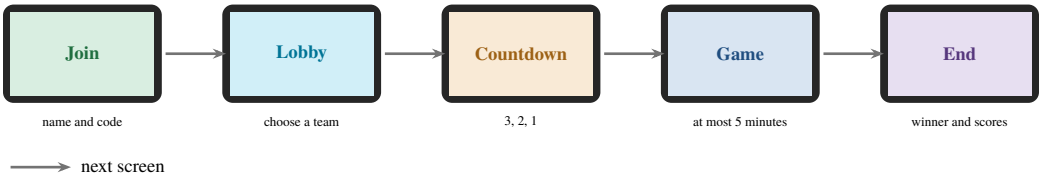
The phone is held sideways. The 3D view fills the screen, and the camera follows the motorcycle from behind. At the top the player sees the time left and the score of each team. In the top corner there is the compass. The arrow points to the delivery point, or to the store when the player has no package. It never shows the distance or the roads. The buttons are large so that the thumbs can find them without looking: left and right on one side, gas and brake on the other side. There is no delivery button: when the player stops at the delivery point, the package is delivered automatically. The “Find me” button asks the server to flash the player’s LED on the board in white for 1 second. After a press, the button is grey and shows the waiting time that M1 puts into the game state.



1 Time left
2 Team scores
3 Compass arrow, no map
4 Own motorcycle, team colour
5 NPC car and an obstacle
6 Left and right
7 Gas and brake
8 “Find me” button

Screens:

The page has five screens. On the join screen the player types a name and joins the game. In the lobby the player chooses one of the four teams and waits. A 3 second countdown starts the game. The game screen is shown above. At the end, the page shows the winner and the scores of all teams.



Smooth play on a real network:

The server sends a state 30 times per second, but Wi-Fi adds delay. If the phone waited for the server, the motorcycle would react late to a button. So the phone moves its own motorcycle at once with the M3 code, and corrects it when the real state arrives. For the other players and the NPC cars, the phone keeps the last few states and draws them a little in the past. Then it can always move them smoothly between two real states, and they do not jump.

The 3D city:

The city is built from the same city plan as the LED board: 6 roads in each direction, 60 road segments and 25 blocks. Tall buildings stand in the centre, houses and trees are outside, and the four team stores in the corners have the team colours. The models are low poly and come from one Blender file. The board uses the same models for 3D printing, so a player can find the same building on the board and on the phone. Objects that repeat, like trees and houses, are drawn as instances, so a mid range phone can draw a smooth picture. During an earthquake the camera shakes, and the shaking is stronger near the centre of the earthquake.

Obstacles and falls in the 3D city:

Obstacles from the game state are drawn on their road: cones and a barrier for road work, crashed cars for an accident, slow cars for heavy traffic and a wet patch for a water puddle. Obstacles come and go during the game, and each change is seen in the 3D city in the same tick as on the board. A fallen motorcycle lies on the road, and the page shows a penalty countdown until M1 lets it stand up again.

8.6 Testing

M4 is tested alone against a fake server that sends recorded game states and answers the join and the controls. We check joining on Android and iPhone, every button and every screen, the compass target, the “Find me” waiting time, reconnecting after a page reload with the same player key, and the prediction with added delay and loss. We also check that a mid range phone draws the full city smoothly.

9. M5 Admin Panel and Monitoring

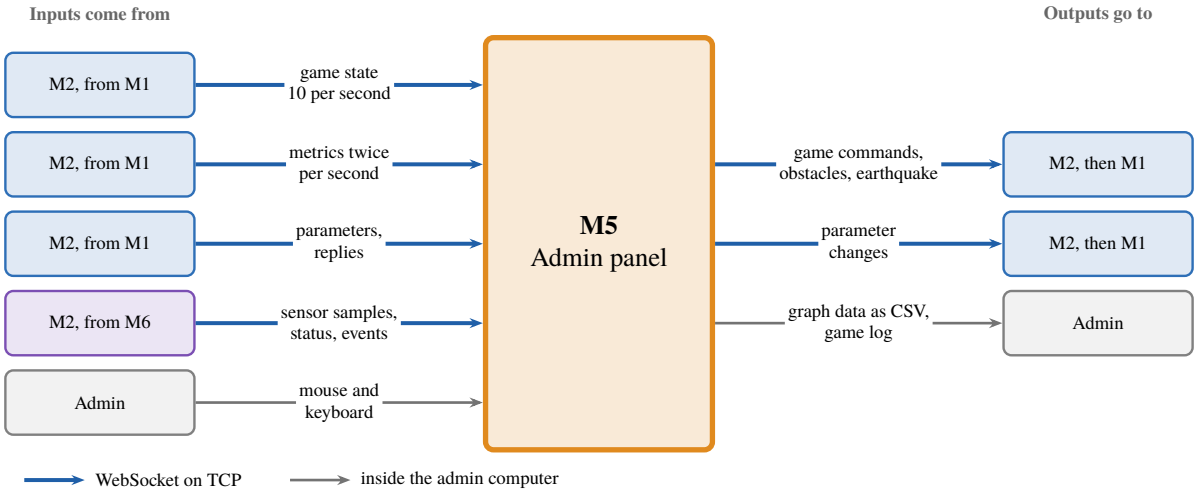
Team: Batuhan Yanar, Ayşe Poyraz, Berk Andaçgil

9.1 Purpose

M5 is the admin screen. On one page the admin starts and controls the game, changes all parameters during play, puts obstacles on a live map, starts earthquakes and watches the results as live graphs. Each change can be seen in three places: in the 3D city on the phones, on the LED board, and in the graphs on this page.

Runs on	Technology	Requirements
Browser on the main computer, or a laptop on the game Wi-Fi. The page comes from M2	TypeScript web page, a light chart library for the live graphs, a 2D drawing of the city for the live map, MessagePack messages over one WebSocket	R6 R7 R9 R10 R11

The admin page talks only to M2, over one WebSocket. The figure shows what the page gets and what it sends.



9.2 Requirements covered

M5 takes part in five requirements. The last column says what will work when M5 is finished.

ID	Requirement	What M5 does for it	When M5 is done
R6	Obstacles	Map tools: a brush for each obstacle type, an eraser, a tool to close a road, an obstacle in front of a player, and the traffic density. Parameters for how often each obstacle type appears and how long it stays	The admin can put any obstacle on any road during the game and can change the rate and the duration of each type
R7	Earthquake	An earthquake panel with a strength control and a live seismic graph of the board sensor	The admin can start an earthquake and compare it with a hand hit
R9	Live admin panel	A form for every parameter group, and graphs with a mark for every change	Every change and its effect are seen on one page
R10	Network and measurement	Ping, jitter, loss and frame rate graphs, and the network settings for each player and the board	The admin can make the network worse and measure the result
R11	Engineering requirements	Works against a fake server with recorded data	M5 can be built before the server is ready

9.3 Inputs and outputs

All data comes through M2. The game state for the map comes less often than to the phones, because the map needs fewer updates.

Direction	Data	From / to	Content
In	Game state	M1, through M2	10 times per second, for the live map
In	Metrics	M1, through M2	Twice per second: scores, delivery times and distances, speeds, falls, traffic, ping, jitter, loss, frame rate, board delay. After a reconnect also the recent history
In	Parameters, replies	M1, through M2	Every parameter with its value and limits, ok or error for each command
In	Board data	M6, through M2	Sensor data, board status, earthquake events
Out	Commands	M2, then M1	Parameter changes, start, pause, resume, end, reset, obstacles, earthquake, new game code
Out	Files	Admin	Graph data as CSV, game log

9.4 Main components

The page is made of five parts. The communication manager is the only part that talks to M2.

Component	Takes	Gives
Communication manager	Messages from M2, commands from the other parts	Decoded data for each part, reconnect and full update after a lost link
Parameter manager	Parameter list with limits, admin input	Forms built from the list, changes sent to M1
Map manager	Game state, clicks on the map	The live map, and obstacle commands with road, lane and position
Graph engine	Metrics, board samples, change marks	Live graphs, the seismic graph, CSV files
Top bar and earthquake panel	Phase, time, board status, button presses	Game commands, earthquake command, the event log

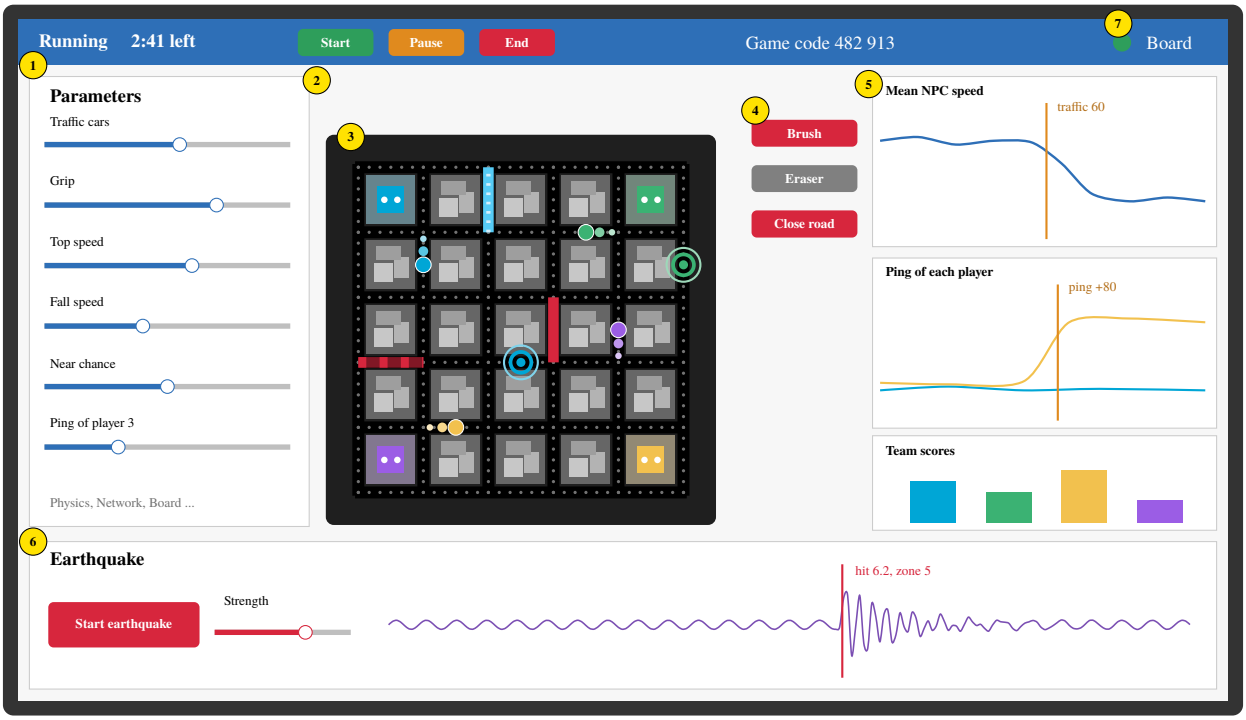
9.5 How it works

The page:

The figure below shows the planned layout. The top bar shows the game phase, the time left, the buttons to start, pause and end the game, the game code and a small light for the board link (green when the board is connected). On the left are the parameter forms. In the centre is the live map: the city from above, drawn again 10 times per second with all motorcycles, NPC cars, obstacles and delivery points. On the right are the live graphs. At the bottom are the earthquake panel and the seismic graph.

Parameters:

The parameters are in groups: game, obstacles, players, physics, earthquake, network (one set for each player) and board link. When the page opens, it asks M1 for all parameters with their limits and builds the forms from this list, so a new parameter needs no new page code. The obstacles group has two values for each of the four types: how often it appears by itself and how long it stays.



1 Game phase, time and buttons
2 Parameter forms
3 Live map of the city
4 Map tools
5 Live graphs with change marks
6 Earthquake panel and seismic graph
7 Board link light

Network settings:

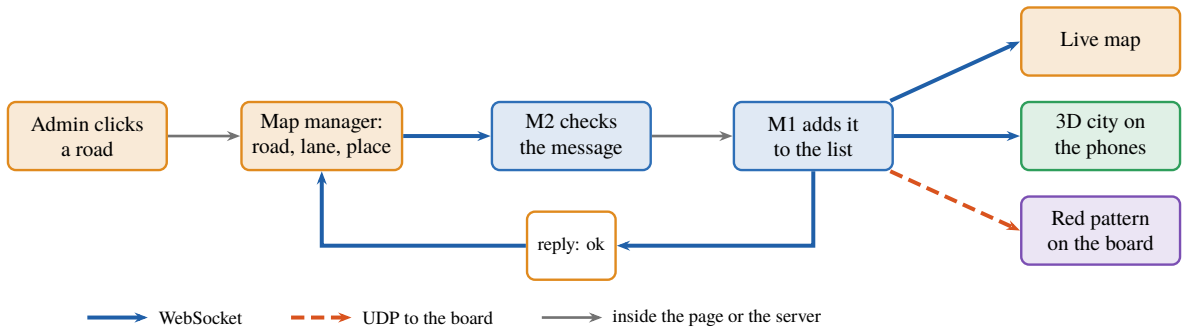
The network group has one row for each player and one for the board. In each row the admin sets an added delay, a jitter and a loss chance, and for a phone also a frame rate limit. M2 applies these values at once. The ping, jitter and loss graphs show the result next to the change mark.

Cause and effect:

The main goal of the panel is to show the effect of a change. So every change puts a thin vertical line on all graphs, with the name of the parameter. For example, when the admin sets more traffic, the line appears and the graph of the mean NPC speed goes down right after it. At the same time the phones show more cars and the board shows more red. The graph data can be saved as a CSV file for the report.

Adding an obstacle:

The admin first chooses a tool, for example the road work brush. Then the admin clicks on a road on the live map. The map manager finds the nearest road segment, the lane and the position on it with the numbering of the city plan. The communication manager sends an “add obstacle” command with the type, road, lane and position. M1 checks that the road exists. Then it adds the obstacle to the list. In the next tick the obstacle is in the game state, so the map, the phones and the board show it at the same time. It stays for the duration of its type, and then M1 removes it. The eraser removes it earlier with a “remove obstacle” command. “Close road” puts road work on a whole segment, and “in front of a player” puts the chosen obstacle a little ahead of that player’s motorcycle.



Earthquake:

The start button sends a command to M1, and M1 asks the board to run its motors. The seismic graph shows the sensor signal of the board live. Each earthquake event puts a mark on the graph with its strength and zone. A hand hit on the board shows on the same graph, so the admin can compare the two.

After a lost link:

If the WebSocket breaks, the page shows a grey bar and tries to connect again. After the new connection, M2 sends the full parameter list, the current game state and the recent history of all graphs that M1 keeps. So the page fills the forms, the map and the graphs again and the admin can go on at once.

9.6 Testing

M5 is tested alone against a fake server that plays back a recorded game with its metrics, parameter list and board data. We check every parameter form with good and wrong values, every map tool, all graphs and the CSV export, a second admin login, and that after a lost link the forms, the map and the graphs are complete again.

10. M6 LED Board and ESP32

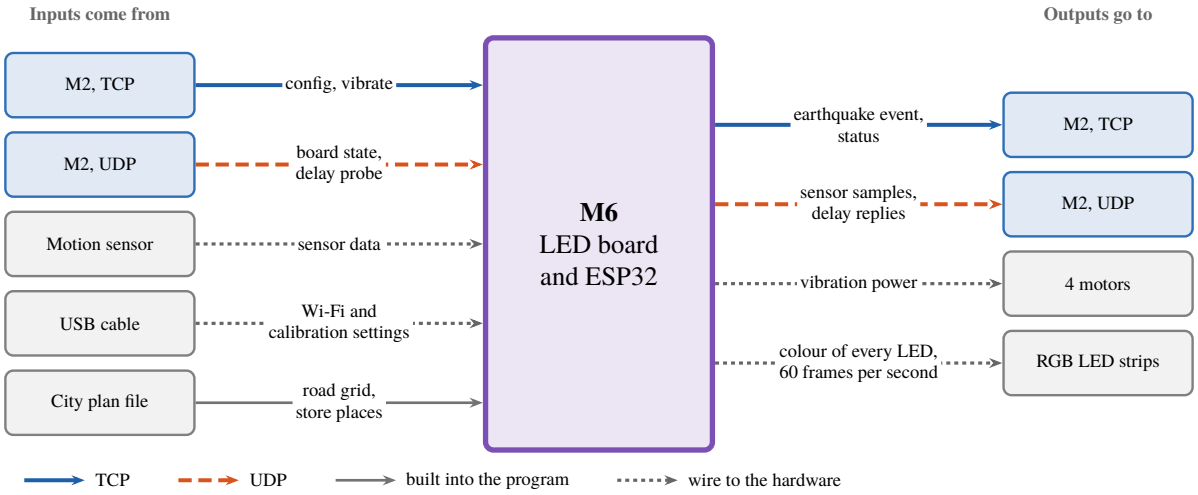
Team: Tuğberk Akbulut, Hasan Alperen Elmas, Hasan Hüseyin Özdemir, Muhib Taha Boy, Ayşe Poyraz, Umutcan Ocak

10.1 Purpose

M6 is the physical city board and the program on its ESP32. The board is 60 × 60 cm. Programmable addressable RGB LED strips lie in all roads and in the four stores, and there are 3D printed buildings, a motion sensor and four vibration motors. In the game the board is the only map, so it must be clear to read from every side of the table. One ESP32 runs one Arduino program with two parts. The link part gets the game state from the main computer over Wi-Fi, finds the strength and the place of a hit on the board, drives the vibration motors and reports the health of the board to the admin. The LED part turns the game state into light.

Runs on	Technology	Requirements
ESP32 in the LED board, and the board itself for the hardware	C++ with Arduino for ESP32, TCP and UDP over Wi-Fi with MessagePack messages, programmable addressable RGB LED strips with an external power supply, a motion sensor with accelerometer and gyroscope, 4 vibration motors, 3D printing, a plywood base	R2 R5 R6 R7 R8 R10 R11

M6 talks to the server over Wi-Fi and to the hardware under the board. The figure shows the data on both sides.



10.2 Requirements covered

M6 takes part in seven requirements. The last column says what will work when M6 is finished.

ID	Requirement	What M6 does for it	When M6 is done
R2	Delivery cycle	Delivery points blink in the player's colour, a flash when the package is delivered	Each player finds the own delivery point on the board
R5	No map on the phone	The board is the map with all motorcycles. "Find me" makes the player's dot white for 1 s	A player can always find the own motorcycle
R6	Obstacles	A red pattern for each type on the whole road segment, light blue for a puddle	Every obstacle type is clear from a distance
R7	Earthquake	Finds hand hits and motor shaking with the sensor, with a strength from 0 to 10 and a zone in a 3 × 3 grid. Drives the motors for the admin button. A shock wave from the earthquake centre	A hit on the table starts an earthquake in the game, and the earthquake is seen on the table
R8	Physical LED board	60 × 60 cm, 6 roads in each direction, LEDs on all roads, printed buildings, four stores. One ESP32 for the LEDs, the sensor and the motors brings the game state over Wi-Fi	The finished board stands on the table and shows the live game without a cable to the computer
R10	Network and measurement	Delay replies, sensor samples for the seismic graph, comes back after a lost link	The admin sees the board delay and the sensor signal live
R11	Engineering requirements	PC emulators for both sides of the link, a demo mode without a server	The server and the board can be built and tested without each other

10.3 Inputs and outputs

Messages that must arrive use TCP, fast and repeated data uses UDP. The board state comes 10 to 30 times per second, but the LED part draws 60 frames per second, so it fills the time between two states by itself.

Direction	Data	From / to	Content
In	Board state	M2, UDP	10 to 30 per second: phase, players (team, colour, position, flags), delivery points, obstacles, earthquake
In	Config	M2, TCP	Player colours, brightness from 0.1 to 1, power limit
In	Vibrate	M2, TCP	Motors to use, strength from 0 to 1, duration
In	Delay probe	M2, UDP	A small message with the server time, answered at once
In	Sensor	Hardware	Motion data from the accelerometer and the gyroscope
In	Settings	USB cable	Wi-Fi name and password, backup server address, calibration values
In	City plan	Shared file	Road grid and store places, turned into the LED map when the code is built
Out	Light	LED strips	The colour of every LED, 60 frames per second
Out	Vibration	4 motors	Motor power with a soft start and safety limits
Out	Earthquake event	M2, TCP, to M1 and M5	Source (hand or motors), strength from 0 to 10, centre, zone from 0 to 8, confidence
Out	Sensor samples	M2, UDP, to M5	Sensor data for the seismic graph, sent in small groups
Out	Status, delay reply	M2	Status on TCP once per second: frame rate, Wi-Fi signal, power estimate, errors. Delay replies on UDP

10.4 Main components

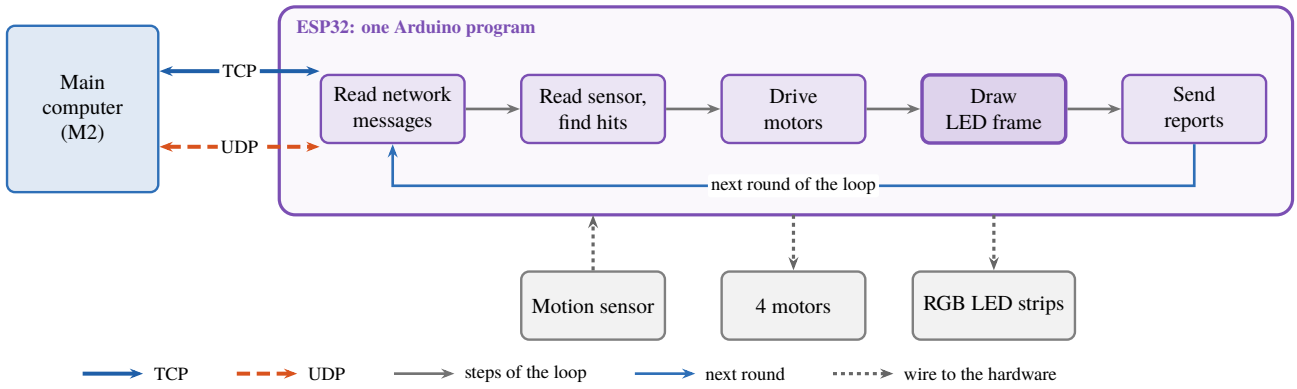
M6 has eleven parts. The first six are the link part and the last five are the LED part. All of them run in the main loop of the Arduino program.

Component	Takes	Gives
Network link	Wi-Fi, server address	One TCP channel and one UDP stream, a new link after a loss
Message codec	Raw messages	Checked messages. Old, broken or wrong version messages are dropped
State hand over	Checked board state and config	The newest full state for the LED part
Hit detector	Sensor samples	Earthquake events with strength, zone and confidence
Motor controller	Vibrate commands	Motor power with a soft start and safety limits
Status and settings	Frame report, USB settings	Status reports, delay replies, settings saved in flash
LED map	City plan	The place of every LED on the roads and in the stores, made by a script
Motion smoother	The last two board states	Smooth motorcycle positions along the roads for every frame
Layers and effects	Board state, phase, link state	Five layers and the patterns for obstacles, earthquakes and game phases
Post processing and output	The mixed frame, config	Corrected colours inside the brightness and power limit, sent to the strips
Demo mode	A built in scenario	A full game look without a server

10.5 How it works

One program with a main loop:

The ESP32 runs one Arduino program. The main loop repeats the same steps again and again: it reads the new network messages, reads the sensor, drives the motors, lets the LED part draw one LED frame and sends the reports. The link part keeps the newest full board state in memory. At the start of each frame the LED part reads this state, so it always draws the latest game.

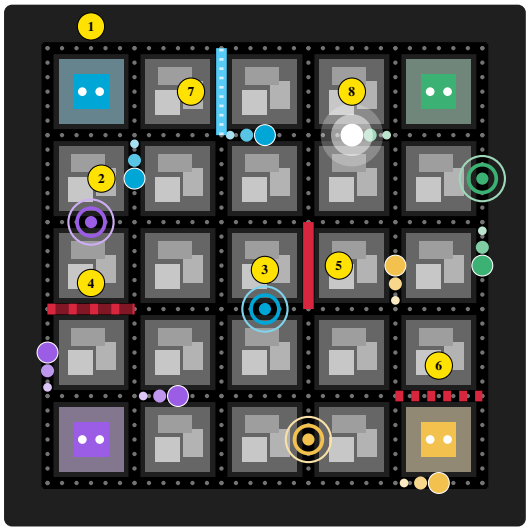


The board:

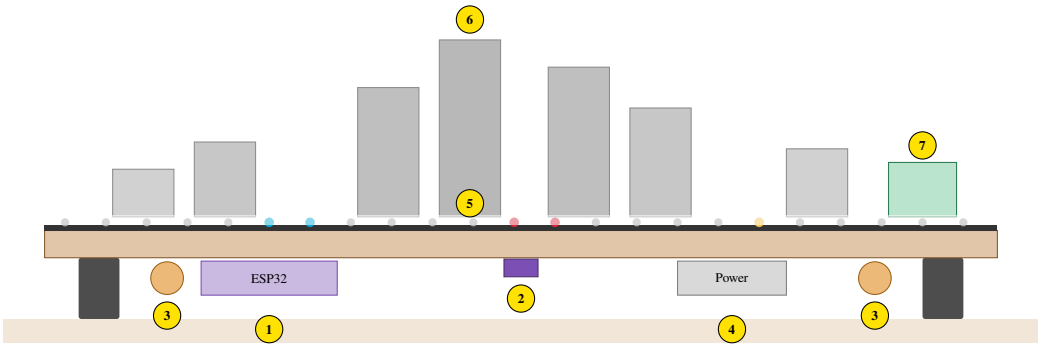
The base is plywood. On it there are 6 roads in each direction, so the city has 5 × 5 blocks. The LED strips lie in the roads between the crossings. There are no LEDs at the crossings, because two strips cannot lie on top of each other. The buildings are 3D printed from the same models as the 3D city on the phone, so a player finds the same building on the board and on the phone. The LEDs need an external power supply, and the software keeps the power under the limit in the config.

What is inside the board:

The figure below is a side cut of the board. The motion sensor is fixed at the centre, under the plywood, so it feels a hit from every side. The four vibration motors are under the four corners. The ESP32 and a power supply for the LEDs and the motors sit under the board too.



1 Team store in the team colour
2 Motorcycle: a bright dot with a short tail
3 Delivery point: blinks in the player's tone
4 Heavy traffic: a slow dim red wave
5 Road work: bright steady red
6 Accident: red blinking
7 Water puddle: light blue shimmer
8 "Find me": white flash for 1 second



1 ESP32
2 Motion sensor at the centre
3 Vibration motors at the corners
4 Power supply
5 RGB LED strips on the roads
6 3D printed buildings
7 Team store with LEDs

Colours:

There are four teams and each team has one colour: blue, green, yellow and purple. The two players of a team have a light and a dark tone of the team colour. Red is used only for obstacles, so red always means "this road slows you down". The pattern of the red tells the type of the obstacle. A water puddle is light blue, because it makes the road slippery, not slow. White is used only for "Find me", so a player always knows that the white dot is the own motorcycle.

Heavy traffic		the light moves slowly along the road
Road work		the whole road is red, it does not move
Accident		red blinks on and off
Water puddle		light blue light that shimmers

Team colours



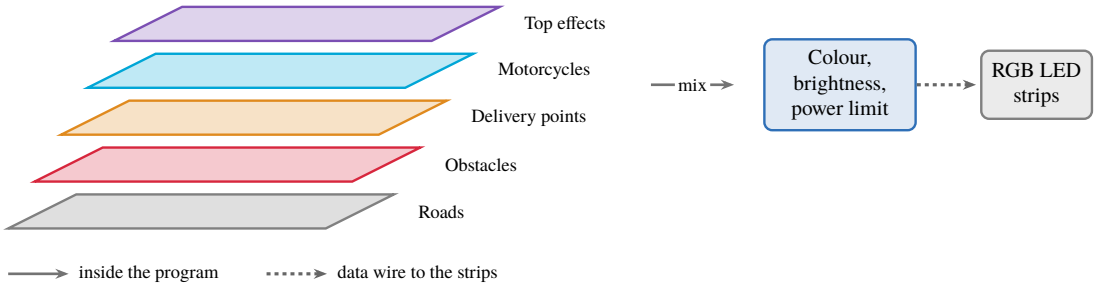
light and dark tone

Layers:

The LED part draws 60 frames per second. Each frame is made from five layers, from the bottom to the top: the roads, the obstacles, the delivery points, the motorcycles and a top layer for short effects. A higher layer covers a lower one, so when two things fall on the same LED, the higher one is shown. When two motorcycles are on the same LED, their colours take turns. After the layers, the LED part corrects the colours for the eye, sets the brightness from the config, and lowers all LEDs a little if the frame would need more power than the limit.

Smooth motion on the roads:

The board state comes less often than the frames, so the motion smoother moves each motorcycle dot between the last two states. It moves the dot along the roads, through the crossing, and never in a straight line over a block. The dot is drawn one state behind the newest one, so it adds a small delay. If a state is lost, the dot keeps moving along its road with its last speed until the next state comes. The dot has a short tail, so the direction of motion is easy to see.



Two channels:

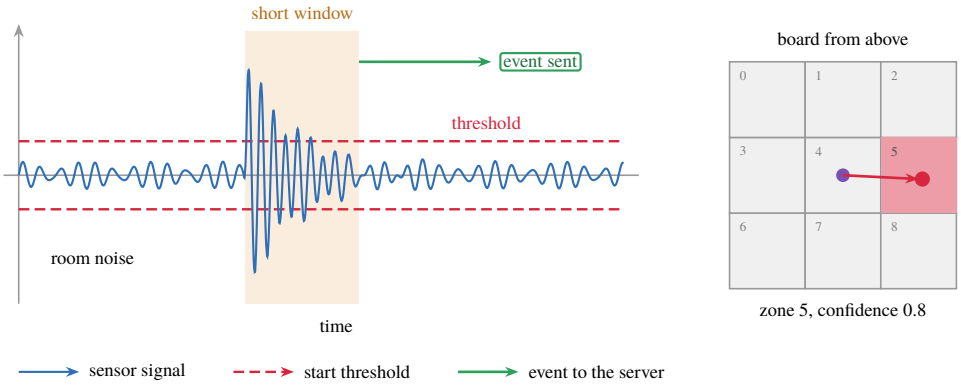
The board keeps one TCP channel and one UDP stream to the server. Messages that must arrive go on TCP: the config, the vibrate commands, the earthquake events and the status report. Each TCP message starts with its length, because TCP is only a stream of bytes. Data that comes again and again goes on UDP: the board state, the sensor samples and the delay probes. If one UDP packet is lost, the next one replaces it, so the LEDs never wait. Every message has an order number, so the board drops a UDP packet that comes late. Each start of the server makes a new session number. When the board sees a new session number, it starts its order numbers again, so it does not drop new messages as old ones.

Finding a hit:

The sensor measures the motion all the time. First a filter takes away gravity and slow tilting, so only the shaking is left. The detector also learns the normal noise of the room and puts the start threshold well above it. So the threshold goes up on a noisy table, and normal play does not start an earthquake. When the signal passes the threshold, a hit starts, and the detector takes a short window from this moment. The stronger the shaking in this window, the higher the strength. Like a real earthquake scale, the strength grows slowly for very strong hits, so a light tap and a hard hit both get a useful value. The scale is set once in calibration with a standard hit, and the result is kept between 0 and 10.

Finding the zone:

The accelerometer shows the direction of the first push of the board, and the gyroscope shows how the board tilts. Together they show the side of the hit, and the size of the push shows how far the hit is from the centre. From this the detector picks one of 9 zones in a 3 × 3 grid and gives a confidence value. After a hit the detector waits a short time, so one hit makes only one event.

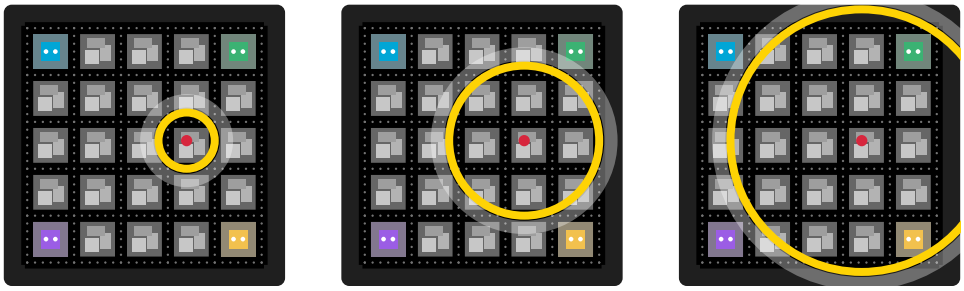


Motors:

The four motors are under the corners, at zones 0, 2, 6 and 8. When the admin presses the earthquake button, M1 sends a vibrate command on TCP. The command says which motors to use, the strength and the time. When the admin chooses one zone, M1 uses the motors nearest to it. For the whole board it uses all four. M6 starts the motors slowly, so the power does not jump. While the motors run, the detector stops learning the room noise, and the whole shaking gives only one event with "motors" as its source. So the earthquake in the game follows the real shaking of the table, and the motors do not start a second earthquake.

Earthquake on the board:

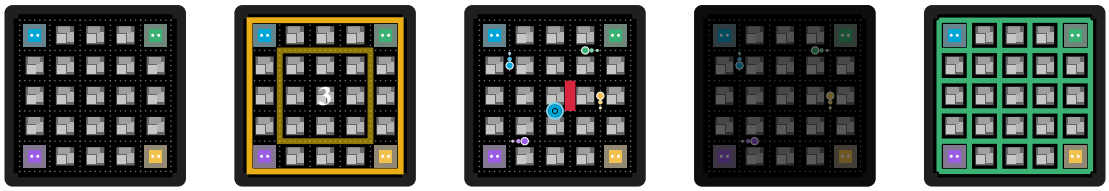
When an earthquake starts, a ring of light grows from the centre of the earthquake over the whole board, like a wave on water, and slowly fades. The road work after an earthquake is not a decision of M6. M1 puts it into the obstacle list, and M6 only draws it like any other obstacle until M1 removes it from the list.



start a moment later later

Game phases and the link:

Every game phase has its own look, so the people around the table always know what happens.



Lobby stores glow, roads dark

Countdown light moves in, 3, 2, 1

Running the live game

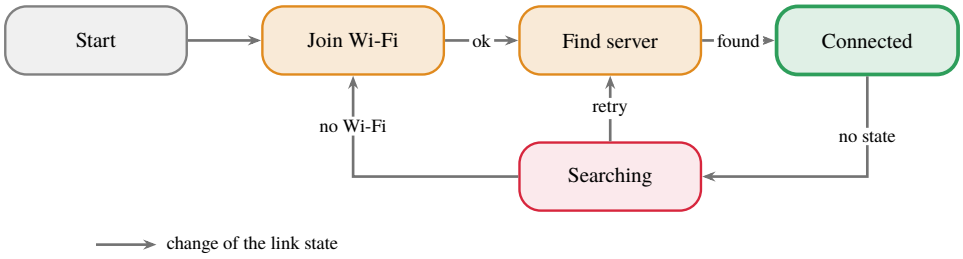
Paused all dims and breathes slowly

Finished roads in the winner colour

In the lobby the roads are dark and the four stores glow in the team colours. In the countdown the light moves from the edge to the centre in three steps, one for each number. During the game the board shows the live state. In a pause everything stays where it is, but all LEDs dim and breathe slowly. At the end all roads turn to the colour of the winner team, and in a draw the colours of the winners take turns.

When the link is lost:

If no state comes for a short time, M6 stops the motors at once and the link state becomes “searching”. The roads go dark and the stores breathe slowly in a dim white, so everybody can see that the board is waiting. Then M6 tries to connect again: first to the server it finds on the local network, then to the backup address. The Wi-Fi name, the password, the backup address and the calibration values are stored in the ESP32’s flash memory. We can change them over the USB cable, so we do not need to build the code again at a new place.



10.6 Testing

M6 is tested without the server in two ways. For the link part two PC programs are used. A server emulator takes the place of M2. It sends test board states, config and vibrate commands with delay and loss that we can change, and it shows the earthquake events and the seismic signal. A board emulator takes the place of the real board, so the server side can also be tested without it. We check the shared example messages, broken packets, a long run, heavy loss, a router restart and a server restart. For the sensor we hit each of the 9 zones many times and check that hits are found, that the zone is right or a neighbour zone, and that the motors do not start a false earthquake. The LED part is tested on the real board with a built in demo scenario. It gives recorded board states as test data and plays a full game with every animation and every game phase. Test patterns check the order and place of the LEDs, the 8 player colours (three people must name each colour correctly), the power limit at full white and the heat in a long run.